

Gold_predictions_pred_final

May 3, 2026

Gold Price Predictions

Project Data [source](#)

```
[ ]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import GradientBoostingRegressor

from sklearn import metrics
```

0.1 Loading the data

```
[ ]: gold_data=pd.read_csv('data/gld_price_data.csv')
```

0.2 exploring the dataset

```
[ ]: gold_data.shape
```

```
[ ]: (2290, 6)
```

```
[ ]: gold_data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 2290 entries, 0 to 2289
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   Date        2290 non-null   str     
 1   SPX         2290 non-null   float64
 2   GLD         2290 non-null   float64
 3   USO         2290 non-null   float64
 4   SLV         2290 non-null   float64
 5   EUR/USD     2290 non-null   float64
```

```
dtypes: float64(5), str(1)
memory usage: 107.5 KB
```

0.3 have Missing values or nulls?

```
[ ]: gold_data.isnull().sum()
```

```
[ ]: Date      0
      SPX      0
      GLD      0
      USO      0
      SLV      0
      EUR/USD   0
      dtype: int64
```

0.4 projecting parts of the data first & last 5 rows respectively

```
[ ]: gold_data.head()
```

```
[ ]:      Date      SPX      GLD      USO      SLV      EUR/USD
0  1/2/2008  1447.160034  84.860001  78.470001  15.180  1.471692
1  1/3/2008  1447.160034  85.570000  78.370003  15.285  1.474491
2  1/4/2008  1411.630005  85.129997  77.309998  15.167  1.475492
3  1/7/2008  1416.180054  84.769997  75.500000  15.053  1.468299
4  1/8/2008  1390.189941  86.779999  76.059998  15.590  1.557099
```

0.5 statistical measurement of the data

```
[ ]: gold_data.describe()
```

```
[ ]:      SPX      GLD      USO      SLV      EUR/USD
count  2290.000000  2290.000000  2290.000000  2290.000000  2290.000000
mean    1654.315776    122.732875    31.842221    20.084997     1.283653
std      519.111540     23.283346    19.523517     7.092566     0.131547
min      676.530029     70.000000     7.960000     8.850000     1.039047
25%     1239.874969    109.725000    14.380000    15.570000     1.171313
50%     1551.434998    120.580002    33.869999    17.268500     1.303297
75%     2073.010070    132.840004    37.827501    22.882500     1.369971
max     2872.870117    184.589996    117.480003    47.259998     1.598798
```

0.6 converting the date into a separate year month day columns , and dropin the old date

```
[ ]:
```

```
[ ]: gold_data['Date'] = pd.to_datetime(gold_data['Date'], format='%m/%d/%Y')
      gold_data['Year'] = gold_data['Date'].dt.year
      gold_data['Month'] = gold_data['Date'].dt.month
```

```
gold_data['DayOfWeek'] = gold_data['Date'].dt.dayofweek
gold_data = gold_data.drop(columns=['Date'])
```

```
[ ]: gold_data.head()
```

```
[ ]:
      SPX      GLD      USO      SLV  EUR/USD  Year  Month  DayOfWeek
0  1447.160034  84.860001  78.470001  15.180  1.471692  2008      1          2
1  1447.160034  85.570000  78.370003  15.285  1.474491  2008      1          3
2  1411.630005  85.129997  77.309998  15.167  1.475492  2008      1          4
3  1416.180054  84.769997  75.500000  15.053  1.468299  2008      1          0
4  1390.189941  86.779999  76.059998  15.590  1.557099  2008      1          1
```

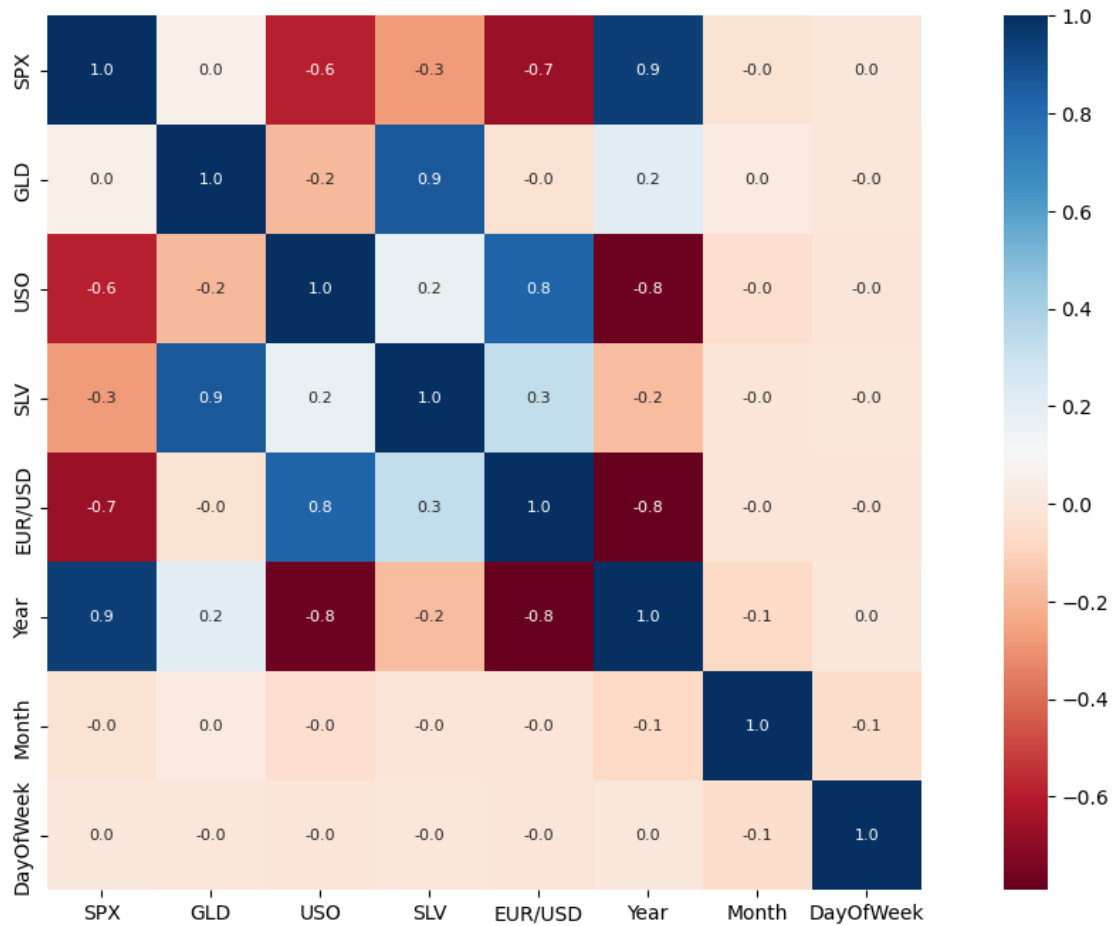
0.7 contracting the correlation between feather

```
[ ]: correlation = gold_data.corr()
```

0.8 visualizing the correlation via a heatmap

```
[ ]: # constructing a heatmap to understand the correlation
plt.figure(figsize = (15,8))
sns.heatmap(correlation, cbar=True, square=True, fmt='.1f', annot=True,
            annot_kws={'size':8}, cmap="RdBu")
```

```
[ ]: <Axes: >
```



0.8.1 correlation of the gold values

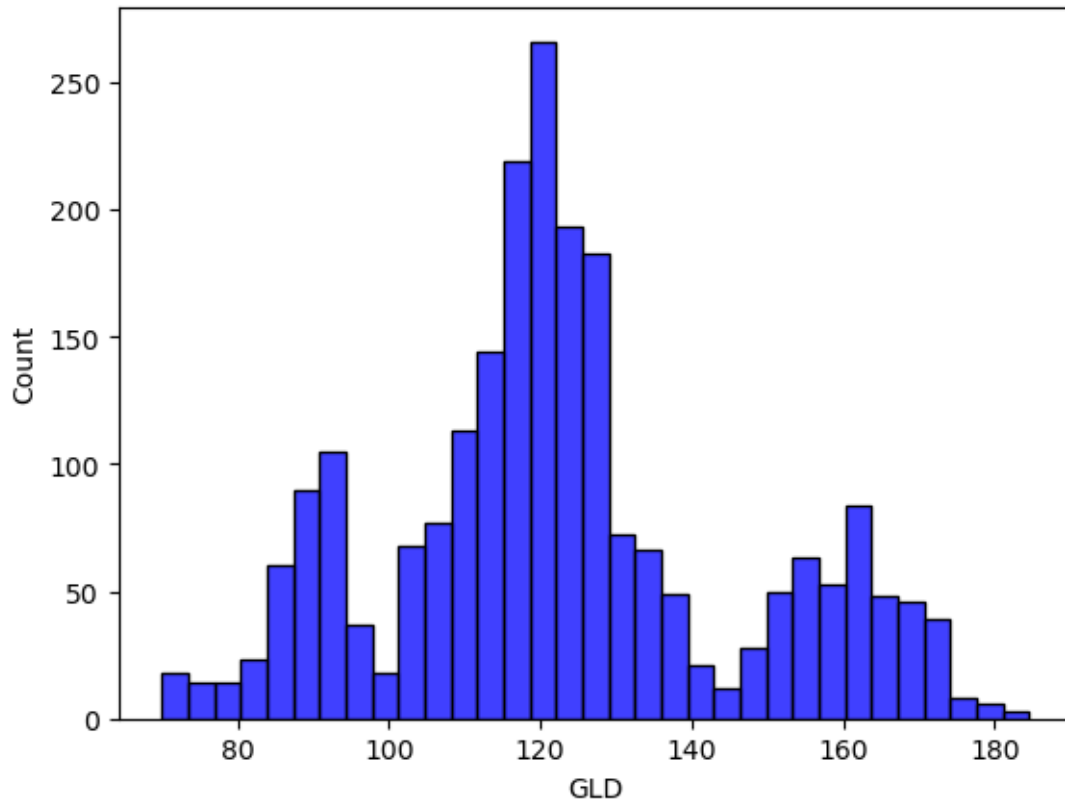
```
[ ]: correlation['GLD']
```

```
[ ]: SPX          0.049345
      GLD          1.000000
      USO         -0.186360
      SLV          0.866632
      EUR/USD     -0.024375
      Year         0.206654
      Month        0.020494
      DayOfWeek   -0.001227
      Name: GLD, dtype: float64
```

0.8.2 checking the distribution of the gld

```
[ ]: sns.histplot(gold_data['GLD'],color="blue")
```

```
[ ]: <Axes: xlabel='GLD', ylabel='Count'>
```



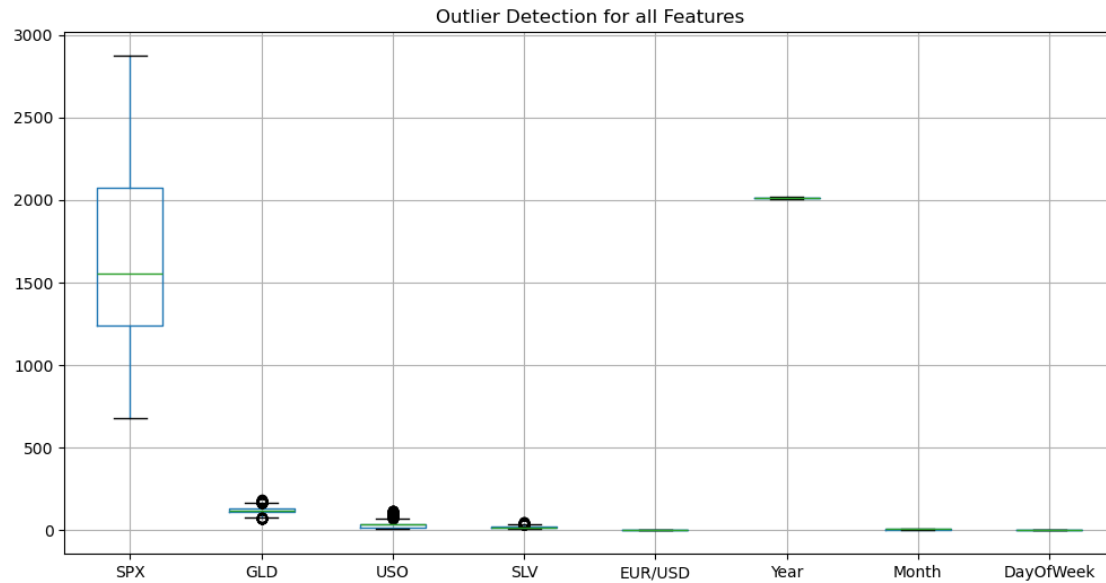
0.8.3 Checking for duplicates

```
[ ]: gold_data.duplicated().sum()
```

```
[ ]: np.int64(0)
```

0.8.4 Outlier detection using Boxplots

```
[ ]: plt.figure(figsize=(12, 6))  
gold_data.boxplot()  
plt.title('Outlier Detection for all Features')  
plt.show()
```



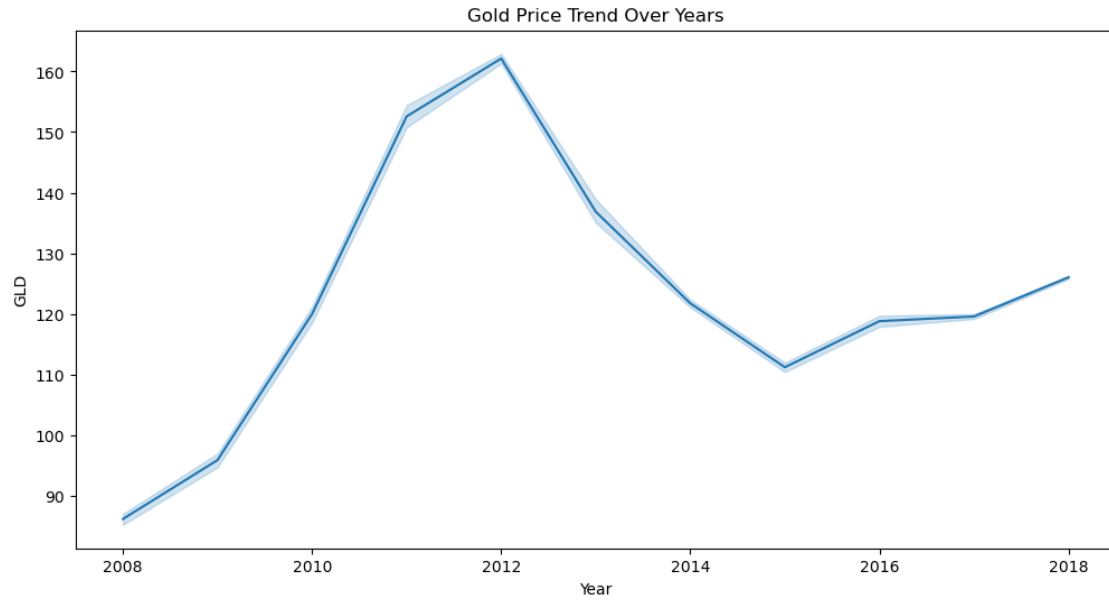
0.8.5 Visualizing the strong relationship between Silver and Gold

```
[ ]: sns.scatterplot(x='SLV', y='GLD', data=gold_data, hue='Year', palette='viridis')
plt.title('Gold Price vs Silver Price')
plt.show()
```



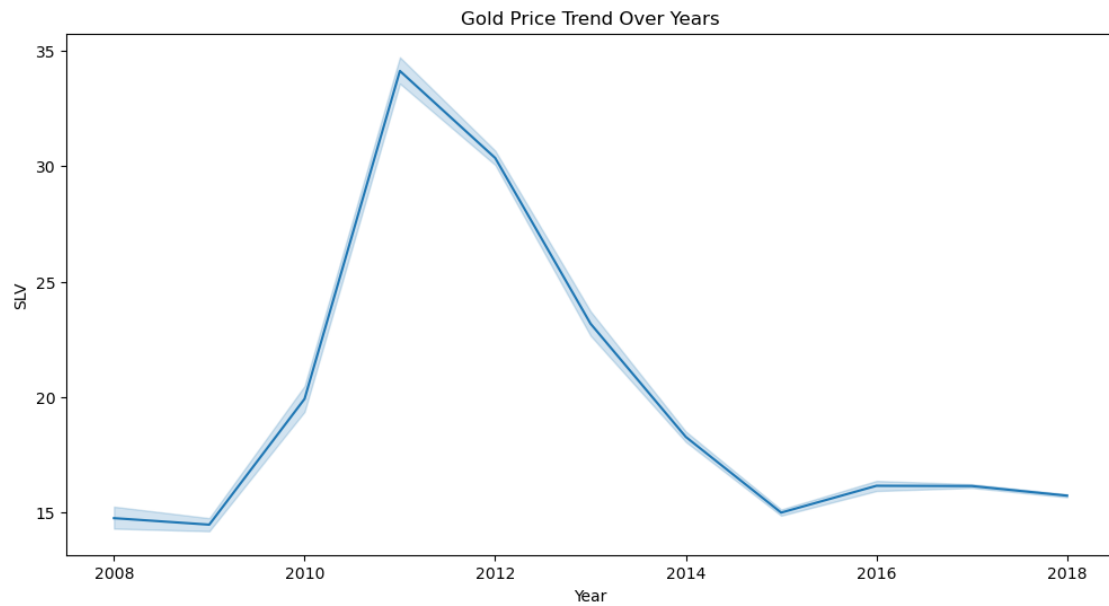
0.8.6 Gold price trend over the years

```
[ ]: plt.figure(figsize=(12, 6))
sns.lineplot(x='Year', y='GLD', data=gold_data)
plt.title('Gold Price Trend Over Years')
plt.show()
```



0.8.7 silver price trend over the years

```
[ ]: plt.figure(figsize=(12, 6))
sns.lineplot(x='Year', y='SLV', data=gold_data)
plt.title('Gold Price Trend Over Years')
plt.show()
```



0.9 preparing the data for the model and removing

0.9.1 We will use today's data to predict tomorrow's gold price & drop the unimportant data month dayofweek

```
[ ]: # Create a new column that pulls tomorrow's Gold price into today's row
gold_data['Target_Next_Day_GLD'] = gold_data['GLD'].shift(-1)

# drop last row
gold_data = gold_data.dropna()

# prepare the target and the training data
y = gold_data['Target_Next_Day_GLD']
X = gold_data.drop(columns=['GLD', 'Target_Next_Day_GLD', 'Month', 'DayOfWeek'])
```

0.10 Splitting the data to test and train

```
[ ]: x_train,x_test,y_train,y_test=train_test_split(X,y,test_size=.2,random_state=2)
```

0.11 Train the models

```
[ ]: randlome_forest=RandomForestRegressor(n_estimators=100)
randlome_forest.fit(x_train,y_train)
```

```
[ ]: RandomForestRegressor()
```

0.12 Model training and prediction

```
[ ]: models = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(n_estimators=100, random_state=2),
    "Gradient Boosting": GradientBoostingRegressor(random_state=2)
}

# 2. Initialize the results dictionary
results = {
    "Model": [],
    "R2 Score": [],
    "RMSE": [],
    "MAE": []
}

print(f"\n{'Model Name':<20} | {'R2 Score':<10} | {'RMSE':<10} | {'MAE':<10}")
print("-" * 60)

# 3. Train and evaluate loop
for name, model in models.items():
    # Train and predict
    model.fit(x_train, y_train)
    predictions = model.predict(x_test)
```

```

# Calculate metrics
r2 = metrics.r2_score(y_test, predictions)
mse = metrics.mean_squared_error(y_test, predictions)
rmse=np.sqrt(mse)
mae = metrics.mean_absolute_error(y_test, predictions)

# Store results
results["Model"].append(name)
results["R2 Score"].append(r2)
results["RMSE"].append(rmse)
results["MAE"].append(mae)

# Print formatted output to console
print(f"{name:<20} | {r2:<10.4f} | {rmse:<10.2f} | {mae:<10.2f}")

```

Model Name	R ² Score	RMSE	MAE

Linear Regression	0.8922	7.58	5.32
Random Forest	0.9882	2.51	1.62
Gradient Boosting	0.9829	3.02	2.03

0.13 see the model performance

```

[ ]: fig, axes = plt.subplots(1, 3, figsize=(16, 5))
x=np.ravel(y_test)
# linear regression
lr=np.ravel(models["Linear Regression"].predict(x_test))
axes[0].set_title("model Liner regression: Actual vs Predicted")
sns.regplot(x=x, y=lr, ax=axes[0],
            scatter_kws={'alpha': 0.5, 'color': 'royalblue'},
            line_kws={'color': 'red', 'linewidth': 2})

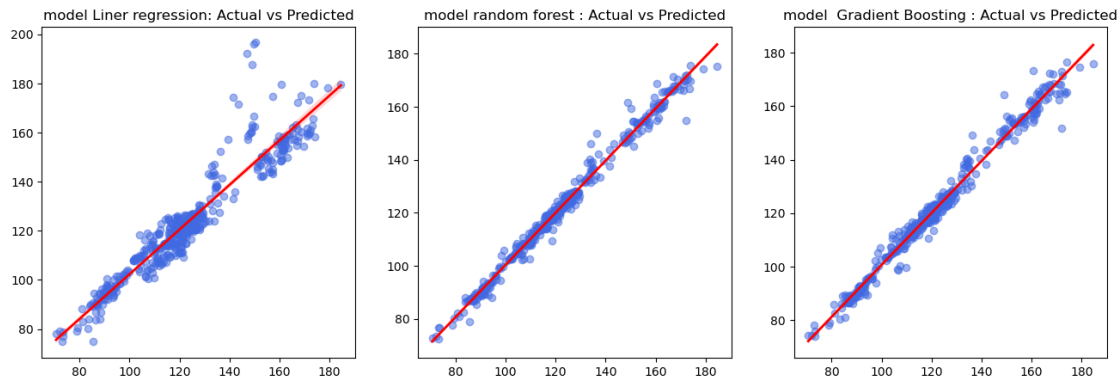
# radome Forest
rf=np.ravel(models["Random Forest"].predict(x_test))
axes[1].set_title("model random forest : Actual vs Predicted")
sns.regplot(x=x, y=rf, ax=axes[1],
            scatter_kws={'alpha': 0.5, 'color': 'royalblue'},
            line_kws={'color': 'red', 'linewidth': 2})

# gradient boost
gb=np.ravel(models["Gradient Boosting"].predict(x_test))
axes[2].set_title("model Gradient Boosting : Actual vs Predicted")
sns.regplot(x=x, y=gb, ax=axes[2],
            scatter_kws={'alpha': 0.5, 'color': 'royalblue'},
            line_kws={'color': 'red', 'linewidth': 2})

```

```
plt.tight_layout()
plt.show();
```

```
[ ]: <Axes: title={'center': 'model Gradient Boosting : Actual vs Predicted'}>
```



0.13.1 converting the result into a dataname for visualization

```
[ ]: results_df=pd.DataFrame(results)
results_df
```

```
[ ]:
```

	Model	R2 Score	RMSE	MAE
0	Linear Regression	0.892210	7.582262	5.324779
1	Random Forest	0.988203	2.508408	1.617456
2	Gradient Boosting	0.982884	3.021420	2.029957

0.13.2 4. Visualize the comparison

```
[ ]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
# --- Subplot 1: R2 Score Accuracy
sns.barplot(data=results_df, x="Model", y="R2 Score", ax=axes[0],
    palette="Blues_d")
sns.barplot(data=results_df, x="Model", y="R2 Score", ax=axes[0],
    palette="Blues_d")
axes[0].set_title("Model Accuracy (R2 Score) - Higher is Better")
# axes[0].set_ylim(0, 1.1)
axes[0].grid(axis='y', linestyle='--', alpha=0.7)
# Subplot 2: Errors MSE and MAE
# We use pd.melt to restructure the dataframe so Seaborn can group the error
    bars side-by-side
error_df = results_df.melt(id_vars="Model", value_vars=["RMSE", "MAE"],
```

```

var_name="Error Type", value_name="Error Value")

sns.barplot(data=error_df, x="Model", y="Error Value", hue="Error Type",
            ax=axes[1], palette="rocket")
axes[1].set_title("Model Errors (RMSE & MAE) - Lower is Better")
axes[1].grid(axis='y', linestyle='--', alpha=0.7)

plt.tight_layout()
plt.show();

```

/tmp/ipykernel_101592/1593784843.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(data=results_df, x="Model", y="R2 Score", ax=axes[0],
palette="Blues_d")

```

/tmp/ipykernel_101592/1593784843.py:4: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.barplot(data=results_df, x="Model", y="R2 Score", ax=axes[0],
palette="Blues_d")

```

